

DLL Development Guide

V5.1.2

Contents

1 Summarize	4
2 API Library Overview	5
2.1 General Structure	5
2.2 Layer 1 :Device Drivers	5
2.2.1 General Description	5
2.2.2 Driver list and description:	6
2.3 Layer 2: Device Common Programming Interface	7
2.3.1 Description	7
2.4 Layer 3:	8
2.4.1 Reader Programming Interface	8
3 Samples Directory	9
4 Development Package Calling Process	10
4.1 Open Reader Driver	10
4.2 Close Reader Driver	10
4.3 RFID Reader Operation	11
4.3.1 Tag Inventory	11
4.3.2 Access tag	11
4.4 NFC APIs	15
4.4.1 Write NDEF Message to Tag	15
4.4.2 Read NDEF Message from Tag	16
4.5 Development of Library Security Doors	16
4.5.1 Development Process Using Automatic Collection	16
4.5.2 Development Process Using RDR_BuffMode_FetchRecords	17
4.5.3 Analysis of Security Door Alarm Record Data Fields	19
4.6 Development of R-PAN/UR-PAN Scanning Mode	19
4.7 Development of JAVA	19
5 Routine Code Parsing	20
5.1 ISO15693 Tag Access C++ Example	20
5.2 ISO14443A-Mifare S50 C++ Example	22
6 Connection String	24
6.1 Description	24
6.2 Build Connection String of Serial Port	24
6.2.1 Format Example	24
6.2.2 Parameter Description	24
6.3 Build Connection String of USB	25
6.3.1 Format Example	25
6.3.2 Parameter Description	25
6.4 Build Connection String of Network	26
6.4.1 Format Example	26
6.4.2 Parameter Description	26
6.5 Build Connection String of Bluetooth	27
6.5.1 Format Example	27
6.5.2 Parameter Description	27

7 Description of Common API Calls for The rfidlib_reader.dll	28
7.1 Enumerate serial ports	28
7.1.1 API Introduction	28
7.1.2 C# programming Example	28
7.2 Enumerate the Inserted USBHID Readers	29
7.2.1 API Introduction	29
7.2.2 C# programming Example	29
7.3 Enumerate the Bluetooth Readers	30
7.3.1 API Introduction	30
7.3.2 C# programming Example	30
7.4 Enumerates the Loaded RFID Reader Driver	31
7.4.1 API Introduction	31
7.4.2 C# Programming Example	31
7.5 RS485 Bus programming	33
7.5.1 Description	33
7.5.2 C++ programming Example(ANRD201)	33
7.6 GPIO Output Control	35
7.6.1 API Introduction	35
7.6.2 API Calling Process	35
6.6.3. C++ programming example	36
7.7 Get Input GPIO Status	37
7.8 Get Device Information	37
7.9 Read/write Configuration Block	37
7.10 To Fetch Buffer Record in Buffer Mode	38
Appendix 1. RFIDLIB Variable Type	39
Appendix 2. Air Protocol Type and Tag Type Coding	46
Appendix 3. RFIDLIB API Error Table	47
Appendix 4: Q&A	49

1 Summarize

According to different RFID reader types we have defined the reader driver interface, each RFID reader has its corresponding driver. Such as the PCI adapter on the computer motherboard.

According to RFID ISO15693, ISO14443B and ISO14443A standards, we define RFID tags access APIs, making the application layer program accessing RFID tags become standardized. In the replacement of the RFID reader, only need to replace the corresponding driver interface, the function of the RFID tag does not need to change.

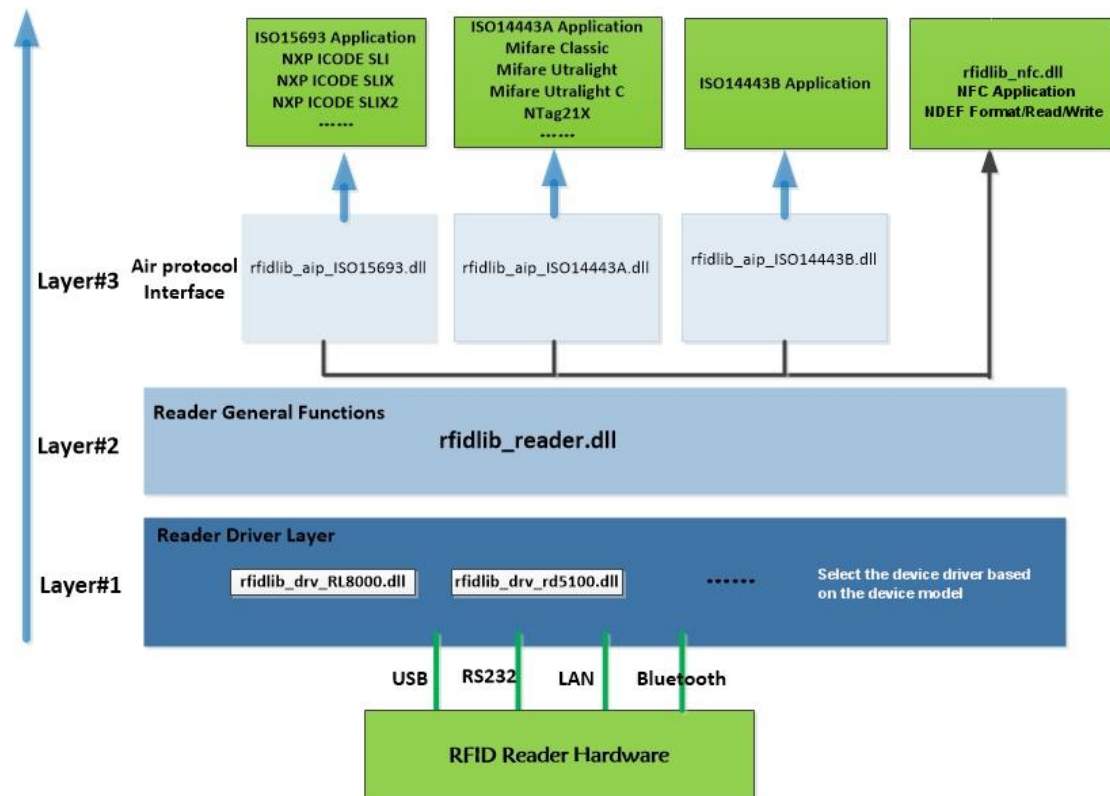
Before using all of the RFIDLIB development kits, you must read the documentation carefully to avoid misuse of the development kit. The document will introduce the structure of RFIDLIB, the connection between the development kit and the calling process.

All documentation is contained in the folder "doc", all development examples contained in the folder "samples", all library DLLs, and exe files for the routines are contained in the samples \ run directory.

2 API Library Overview

2.1 General Structure

RFIDLIB is mainly composed of reader driver programming interface, universal reader programming interface, air interface protocol programming interface, tag reading and writing programming interface, etc.



2.2 Layer 1 :Device Drivers

2.2.1 General Description

Each device model has its corresponding device driver library, you need to develop the corresponding device, just put the corresponding device driver library files are copied to the directory called "Drivers", then when the application is initialized, calls "RDR_LoadReaderDrivers" API, load device drivers into the application, and then use the "RDR_Open" you can open the device driver, and the specific call flow, see "Section 4 Development package calling process".

2.2.2 Driver list and description:

Driver DLL Name	Description
rfidlib_drv_RL8000.dll	RFID reader applicable to this driver including: 1. RL8600 series: RL863, RL865, RL86P ,RL866 。 2. RL1700 series: RL173, RL175, RL17P ,RL176 。 3. RL1500 series: RL153
rfidlib_drv_RD5100.dll	RFID reader applicable to this driver including: RD5101,RD5104,RD5112, RD5124, RD5130,RD503 ,RD543
rfidlib_drv_rpan.dll	RFID reader applicable to this driver including: R-PAN\UR-PAN
rfidlib_drv_RD242.dll	RFID reader applicable to this driver including: RD242,RD201
rfidlib_drv_M201.dll	RFID reader applicable to this driver including: M201
rfidlib_drv_RD5200.dll	RFID reader applicable to this driver including: M22,RD2100,RD3000,RD5200
rfidlib_drv_LSGContro lCenter.dll	RFID gate applicable to this driver including: LSG428,LSG422,LSG425,LSG618
rfidlib_LSG405.dll	RFID gate applicable to this driver including: LSG405,LSG406
rfidlib_drv_UM200.dll	RFID UHF reader applicable to this driver including: ER60M2\UM201\URD5118\URD2004\URD6116\URD8104\UR D1101\UM1101\UM1201\UM2101

2.3 Layer 2: Device Common Programming Interface

2.3.1 Description

The function package abstracts interface functions suitable for all card readers, including tag inventory, tag connection, tag data reading and writing, tag disconnection, setting output port, obtaining input port status, obtaining device information, writing device configuration block, reading device configuration block and so on. Minimize the amount of work developers need to port code to a new card reader. **This function package is a must.**

Library Name	Description
rfidlib_reader.dll	Device common programming interface

2.4 Layer 3:

2.4.1 Reader Programming Interface

2.4.1.1 Description

The function package is based on the device common programming interface “rfidlib_reader.dll” that defines a common API of the air interface protocol, such as the command for the tag operation of the protocol, and the various inventory parameters of the protocol.

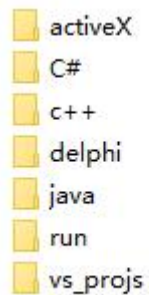
2.4.1.2 Supported Air Protocol Interface

Note: Select the corresponding programming interface according to different air interface protocols.

Name	Description
rfidlib_aip_iso15693.dll	ISO15693 protocol related interface, used to create the parameters of the Inventory of ISO15693 and parse the tag data information of the Inventory, and the operation function of all the labels of ISO15693.
rfidlib_aip_iso14443A.dll	ISO14443A protocol related interface, used to create the parameters of the Inventory of ISO14443A and parse the tag data information of the Inventory, and the operation function of all the labels of ISO14443A.
rfidlib_aip_iso18000p3m3.dll	ISO18000-3 mode 3 protocol related interface.

3 Samples Directory

Samples directory



3.3.1.vs_projs

Vs2005 solution sln, the solution contains all the c++ and c# project

3.3.2.c++

This directory contains **MFC** development samples

3.3.3.C#

This directory contains **C#** development samples

3.3.4.delphi

This directory contains **delphi** development samples

3.3.5.java

This directory contains **JNI** development samples

3.3.6.activeX

This directory contains **OCX** development samples

3.3.7.run

The directory contains c++, C#, delphi example of the project to generate the final executable file exe and API dll file.

4 Development Package Calling Process

4.1 Open Reader Driver

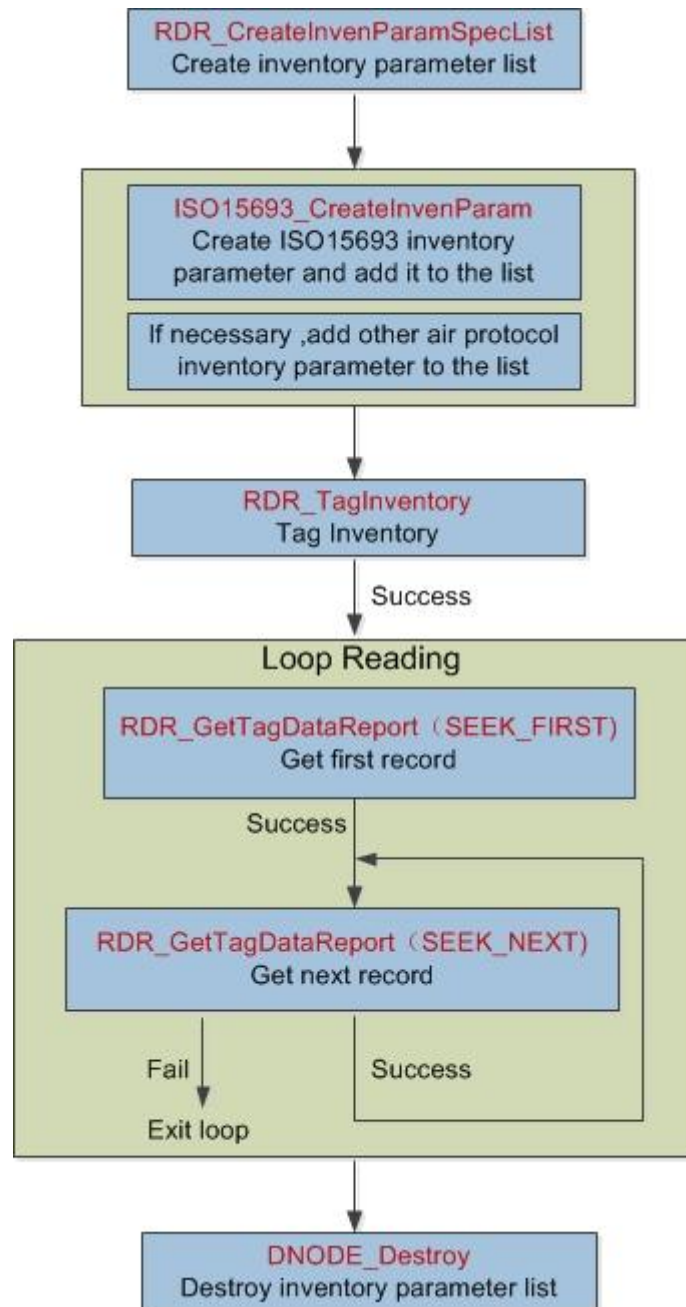
Step 1	Copy the required DLL files and directories from "Samples\Run" to the directory where the application test executable is output
Step 2	Calling RDR_LoadReaderDrivers("\\Drivers") while the application is loading, The API will load all device driver DLLS (e.g. rfidlib_drv_RL8000.dll,rfidlib_drv_rd5100.dll, etc.) in all specified directories. For details, see the documentation for rfidlib_reader.dll.
Step 3	Call RDR_Open, pass the connection string of the driver (see 5.0), open the communication interface and create the instance of the corresponding driver. All the following operations need to pass the handle of the instance, see the description of rfidlib_reader.dll for details.

4.2 Close Reader Driver

Name	Description
RDR_Close	Close the device driver, see the instructions for rfidlib_reader.dll

4.3 RFID Reader Operation

4.3.1 Tag Inventory



4.3.2 Access tag

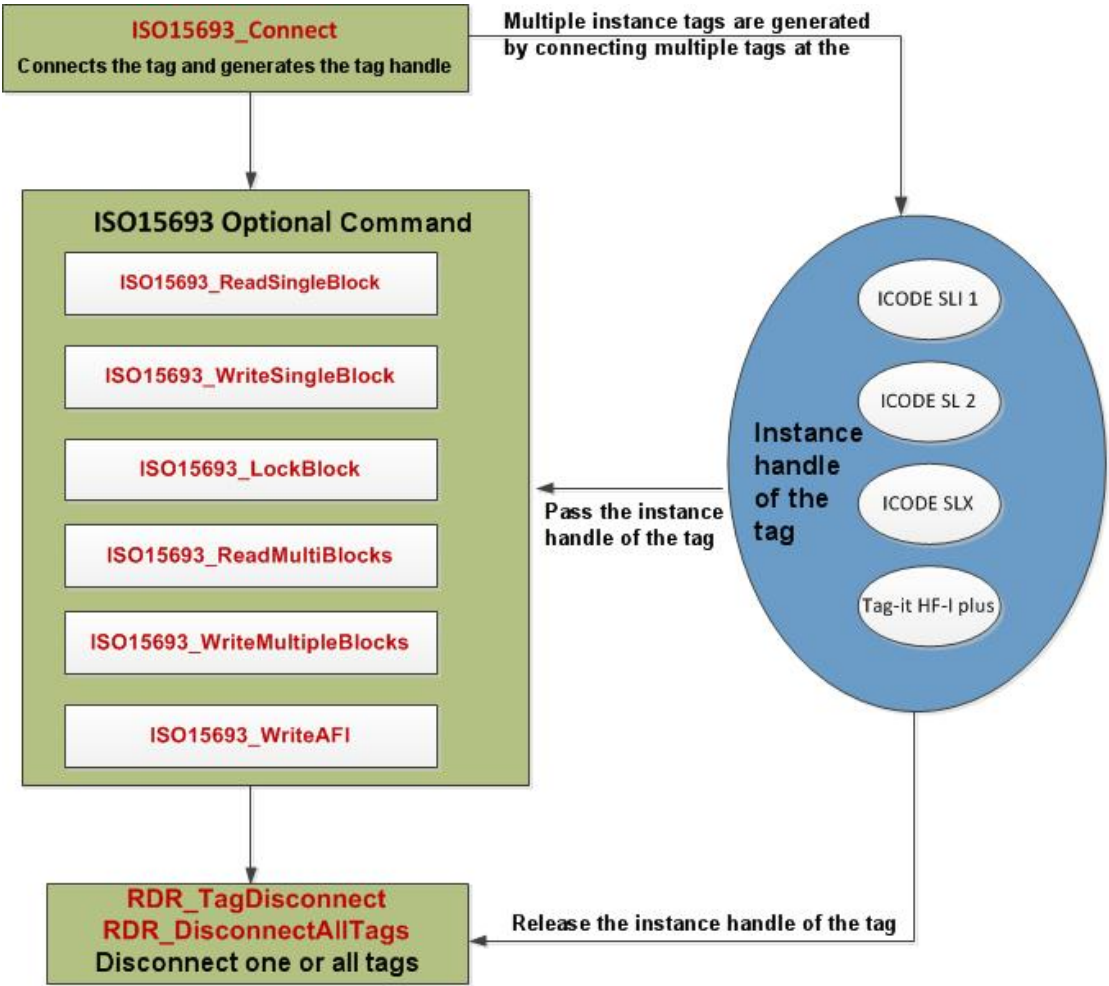
Note:

For readers that support multiple antenna interfaces, the API "RDR_SetAccessAntenna" (in

rfidlib_reader.dll) is called to select the antenna interface before manipulating the tag. The tag inventory tells the application the antenna number where the tag is located.

4.3.2.1 Access ISO15693 tag

Flow chart:



Detailed step description:

Step	Description	API name
1	Connect Tag (multiple tags can be connected simultaneously)	ISO15693_Connect , For detail see "rfidlib_aip_iso15693.doc"

2	ISO15693 Optional commands	ISO15693_ReadSingleBlock ISO15693_WriteSingleBlock ISO15693_LockBlock ISO15693_ReadMultiBlocks ISO15693_WriteMultipleBlocks ISO15693_WriteAFI ISO15693_LockAFI ISO15693_WriteDSFID ISO15693_LockDSFID ISO15693_GetSystemInfo ISO15693_GetBlockSecStatus For detail see "rfidlib_aip_iso15693.doc"
	NXP ICODE SLI Custom commands	NXPICODESLI_EableEAS NXPICODESLI_DisableEAS NXPICODESLI_LockEAS NXPICODESLI_EASAlarm For detail see "rfidlib_aip_iso15693.doc"
	NXP ICODE SLIX Custom commands	NXPICODESLIX_EableEAS NXPICODESLIX_DisableEAS NXPICODESLIX_LockEAS NXPICODESLIX_EASAlarm NXPICODESLIX_GetRandomNum NXPICODESLIX_SetPassword NXPICODESLIX_WritePassword NXPICODESLIX_LockPassword NXPICODESLIX_PasswordProtect For detail see "rfidlib_aip_iso15693.doc"
	Tag-it HF-I plus Custom commands	TIHFIPLUS_Write2Blocks TIHFIPLUS_Lock2Blocks For detail see "rfidlib_aip_iso15693.doc"
3	Tag disconnect / All tag disconnect See the development kit reader_general for the API	RDR_TagDisconnect RDR_DisconnectAllTags, For detail see "rfidlib_reader.doc"

4.3.2.2 Access Mifare Ultralight

1. Mifare ultralight operation invocation process

Step	Description	API
1	Connect tag	ULTRALIGHT_Connect, For detail, please see "rfidlib_aip_iso14443a.doc"

2	Data page access commands	ULTRALIGHT_Read ULTRALIGHT_Write ULTRALIGHT_LockPage ULTRALIGHT_LockBlock ULTRALIGHT_ReadMultiplePages ULTRALIGHT_WriteMultiplePages ULTRALIGHT_Authenticate ULTRALIGHT_UpdatePassword ULTRALIGHT_UpdateAuthConfig For detail, please see "rfidlib_aip_iso14443a.doc"
3	Disconnect tag or all tags	RDR_TagDisconnect RDR_DisconnectAllTags For detail, please see "rfidlib_reader.doc"

4.3.2.3 Access Mifare classic (including S50 and S70)

1. API description

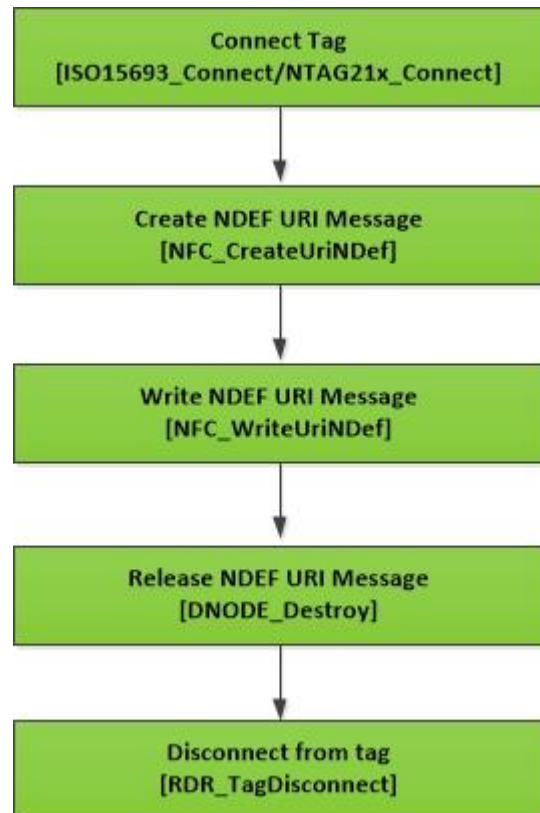
Step	Description	API
1	Connect tag	MFCL_Connect, For detail, please see "rfidlib_aip_iso14443a.doc"
2	KeyA or KeyB authentication	MFCL_Authenticate, For detail, please see "rfidlib_aip_iso14443a.doc"
3	Data block access	MFCL_ReadBlock MFCL_WriteBlock MFCL_Increment MFCL_Decrement MFCL_Restore MFCL_Transfer MFCL_FormatValueBlock For detail, please see "rfidlib_aip_iso14443a.doc"
4	Disconnect tag or all tags	RDR_TagDisconnect RDR_DisconnectAllTags For detail, please see "rfidlib_reader.doc"

4.3.2.4 ISO 18000-6C (UHF)

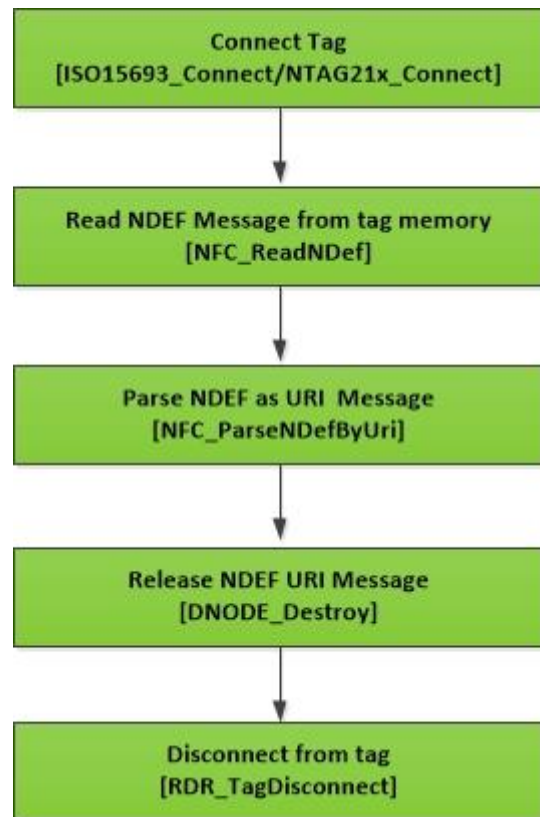
Wait for updating

4.4 NFC APIs

4.4.1 Write NDEF Message to Tag



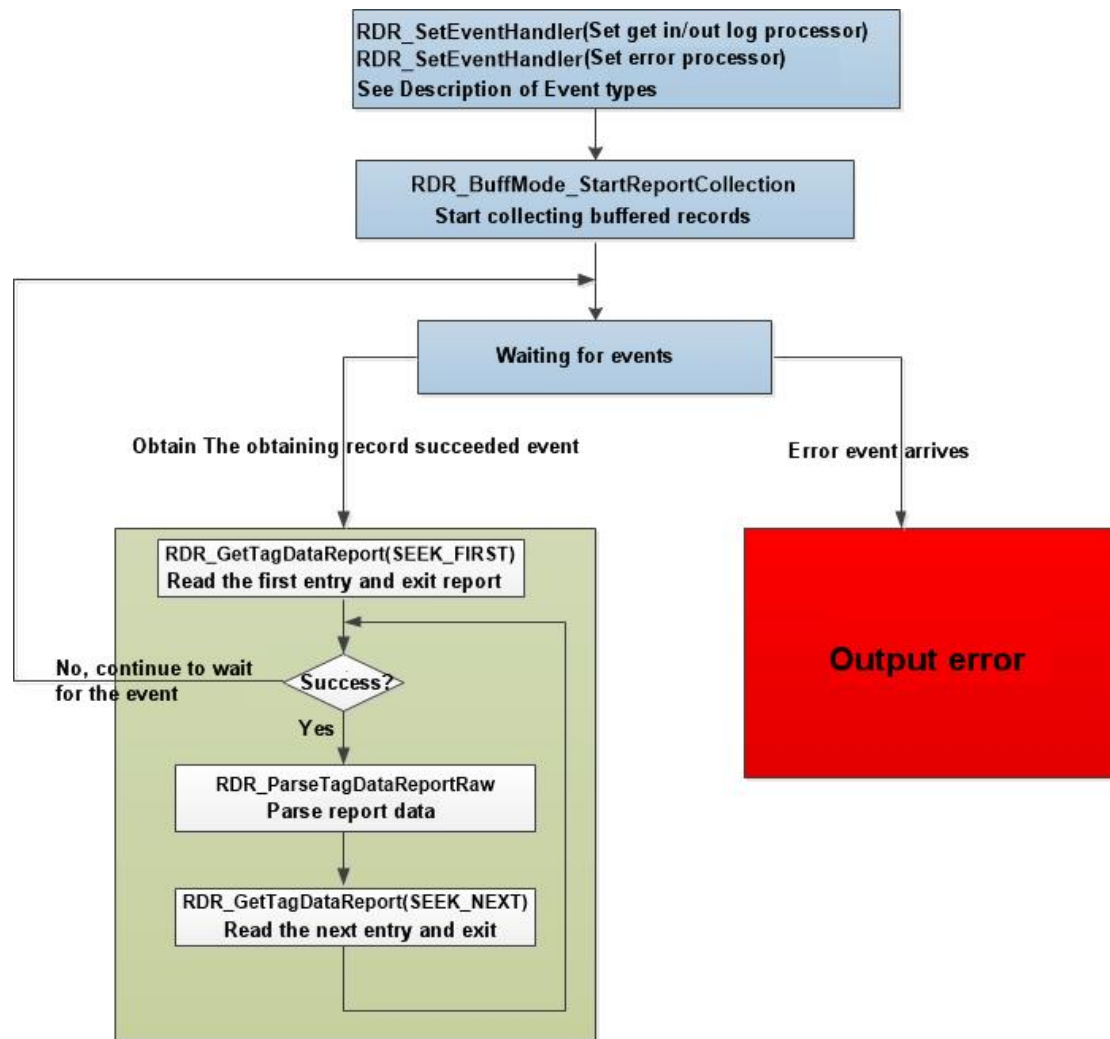
4.4.2 Read NDEF Message from Tag



4.5 Development of Library Security Doors

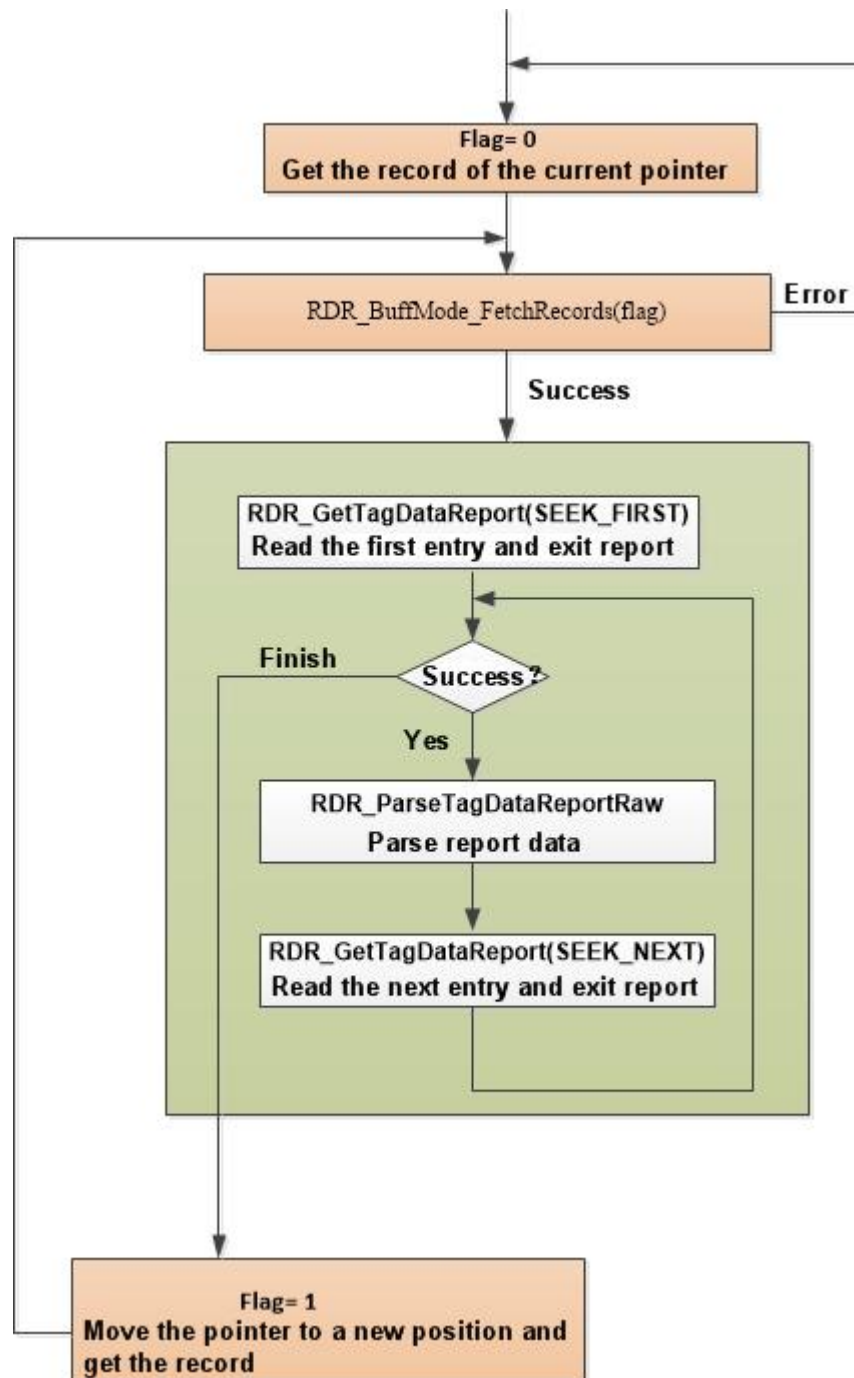
4.5.1 Development Process Using Automatic Collection

A thread is created in the library that automatically calls the function `RDR_BuffMode_FetchRecords` and sends the `RFIDLIB_EVNT_GETBUF_REPORT_FOUND` event to the application when the record is fetched, and if there is an error, an error event is sent.



4.5.2 Development Process Using RDR_BuffMode_FetchRecords

To use this function, the application needs to create its own thread, loop `RDR_BuffMode_FetchRecords`. There is no need to use functions such as `RDR_SetEventHandler`, `RDR_BuffMode_StartReportCollection`, `RDR_BuffMode_StopReportCollection` etc.



4.5.3 Analysis of Security Door Alarm Record Data Fields

RDR_ParseTagDataReportRaw returns the original data of the record.

A. Record structure for alarm and entry/exit record types:

Record header	Fields	Data type	Description	
	Record type	BYTE	CODE	Description
			0x00	EAS alarm record
			0x01	AFI1 alarm record
			0x02	AFI2 alarm record
			0x03	AFI3 alarm record
			0x11	DSFID1 alarm record
			0x12	DSFID2 alarm record
			0x13	DSFID3 alarm record
			0x21	Reader's card AFI1 Entry/Exit Record
			0x22	Reader's card AFI2 Entry/Exit Record
			0x33	Reader's card AFI3 Entry/Exit Record
	Entry/exit direction	BYTE	1: Entry, 2: Exit	
Time	BYTE[6]	BCD code of year, month, day, hour, minute and second . B[0]: 2 bits, year, such as 2016 then the value is 0x16; B [1]: Month; B [2]: Day; B [3]: hour; B [4]: points; B [5]: seconds;		
Card data length	BYTE	The length of the card content included		
Card data	BYTE [Card Data Length], currently containing 8 bytes of UID or block data			

4.6 Development of R-PAN/UR-PAN Scanning Mode

This part of the content needs to be supplemented.

4.7 Development of JAVA

This part of the content needs to be supplemented.

5 Routine Code Parsing

5.1 ISO15693 Tag Access C++ Example

1. Load all reader drivers including in the “Drivers”

Note: This function only needs to run once after the software starts

```
RDR_LoadReaderDrivers("\\Drivers"); //
```

2. Open reader driver

```
RFID_READER_HANDLE hr;
```

```
//Build connection string
```

```
CString connstr="RDType=RD122;CommType=USB;AddrMode=0;SerNum=";
```

```
int iret = RDR_Open(connstr,&hr) ;
```

```
If(iret ==0) {
```

```
    //open ok
```

```
}
```

```
Else {
```

```
    //open fail
```

```
}
```

3. Inventory and Read

```
//tell RDR_TagInventory we want to read ISO15693 tag
```

```
RFID_DN_HANDLE dnInvenParamList= RFID_HANDLE_NULL ;
```

```
dnInvenParamList = RDR_CreateInvenParamSpecList();
```

```
if(dnInvenParamList){
```

```
    ISO15693_CreateInvenParam (dnInvenParamList,0) ;
```

```
}
```

```
//send inventory command
```

```
iret = RDR_TagInventory(hr,1,0,NULL ,dnInvenParamList) ;
```

```
if(iret == NO_ERR) {
```

```
    //inventory OK, and try to get all the tag report from local memory .
```

```
RFID_DN_HANDLE dnhReport = RDR_GetTagDataReport(hr,RFID_SEEK_FIRST) ;
```

```
while(dnhReport != RFID_HANDLE_NULL)
```

```
{
```

```
    DWORD AIptype,TagType,AntId ;
```

```
    BYTE dsfid;
```

```
    BYTE uid[8];
```

```
    BYTE uidlen;
```

```
    iret = ISO15693_ParseTagDataReport
```

```
(dnhReport,&AIptype,&TagType,&AntId,uid,&uidlen) ;
```

```
if(iret == NO_ERR) {
```

```
    //try to read data blocks from tag
```

```
    RFID_TAG_HANDLE hp;
```


5.2 ISO14443A-Mifare S50 C++ Example

1. Load all reader drivers including in the “Drivers”

Note: This function only needs to run once after the software starts

```
RDR_LoadReaderDrivers("\\Drivers"); //
```

2. Open reader driver

```
RFID_READER_HANDLE hr;
```

```
//Build connection string
```

```
CString connstr="RDType=RD122;CommType=USB;AddrMode=0;SerNum=";
```

```
int iret = RDR_Open(connstr,&hr) ;
```

```
If(iret ==0) {
```

```
    //open ok
```

```
}
```

```
Else {
```

```
    //open fail
```

```
}
```

3. Inventory and Read data block

```
//tell RDR_TagInventory we want to read ISO14443A tag
```

```
RFID_DN_HANDLE dnInvenParamList= RFID_HANDLE_NULL ;
```

```
dnInvenParamList = RDR_CreateInvenParamSpecList();
```

```
if(dnInvenParamList){
```

```
    ISO14443A_CreateInvenParam(dnInvenParamList,0) ;
```

```
}
```

```
//send inventory command
```

```
iret = RDR_TagInventory(hr,1,0,NULL ,dnInvenParamList) ;
```

```
if(iret == NO_ERR) {
```

```
    //inventory OK, and try to get all the tag report from local memory .
```

```
RFID_DN_HANDLE dnhReport = RDR_GetTagDataReport(hr,RFID_SEEK_FIRST) ;
```

```
while(dnhReport != RFID_HANDLE_NULL)
```

```
{
```

```
    DWORD AIptype,TagType,AntId ;
```

```
    BYTE dsfid;
```

```
    BYTE uid[8];
```

```
    BYTE uidlen;
```

```
    iret =
```

```
    ISO14443A_ParseTagDataReport(dnhReport,&AIptype,&TagType,&AntId,uid,&uidlen) ;
```

```
if(iret == NO_ERR) {
```

```
    RFID_TAG_HANDLE hp;
```

```
    iret = MFCL_Connect(hr,0,uid,&hp);
```

```
    if(iret == 0) {
```

```
        BYTE keyType=0; //0: KEY A ,1:KEY B
```

```
        BYTE key[6]={0xff,0xff,0xff,0xff,0xff,0xff} ;
```

```

        iret = MFCL_Authenticate(hr, hp, 44, keyType, key);
        if(iret == 0) {
            BYTE blockBuffer[16];
            iret = MFCL_ReadBlock(hr, hp, 44, blockBuffer, sizeof(blockBuffer));
            if(iret == 0) {
                //Read data block OK ,add your code here

            }
        }
        //Disconnect from tag
        RDR_TagDisconnect (ht, hp);
    }
}
dnhReport = RDR_GetTagDataReport(hr, RFID_SEEK_NEXT);
}
}
Else{
    //fail
}

```

```

if(dnInvenParamList) DNODE_Destroy(dnInvenParamList);

```

4. Close reader driver

```

RDR_Close(hr);

```

6 Connection String

6.1 Description

The connection string is made up of parameter fields, separated by a ";" semicolon. Each parameter has a name and a value separated by "=". The connection string has two basic parameters "RDType" (Reader driver type) and "CommType" (communication interface type). Please refer to the document "Quick Start to Device Development" for the name of the device driver.

6.2 Build Connection String of Serial Port

6.2.1 Format Example

"RDType=RD201;
CommType=COM;COMName=COM1;BaudRate=38400;Frame=8E1;BusAddr=255"

6.2.2 Parameter Description

No	Name	Parameter Value	
1	RDType	Please refer to "Quick Start to Device Development"	
2	CommType	Communication interface type. Set "CommType=COM"	
		Value	Description
		COM	Serial port
		USB	USB interface
		NET	Network interface
		BLUETOOTH	Bluetooth interface
3	COMName	COM name, Such as "COM1", "COM2" ...	
4	BaudRate	Baud rate ,such as "38400"	
5	Frame	Frame type:	
		Value	Description
		"8E1"	8 data bits, Even, 1 stop bit
		"8O1"	8 data bits, Odd, 1 stop bit
		"8N1"	8 data bits, None, 1 stop bit
6	BusAddr	RS485 Bus address, 1-254, 255 is broadcast address.	

6.3 Build Connection String of USB

6.3.1 Format Example

“RDType=M201;CommType=USB;AddrMode=1;SerNum=1000011”

6.3.2 Parameter Description

No	Name	Parameter value	
1	RDType	Please refer to “Quick Start to Device Development”	
2	CommType	Communication interface type. Set “CommType=USB”	
		Value	Description
		COM	Serial port
		USB	USB interface
		NET	Network interface
		BLUETOOTH	Bluetooth interface
3	AddrMode	Address mode:	
		Value	Description
		0	None address, automatically search the first device has been inserted USB interface, and open. If more than one device is searched for, a failure is returned.
		1	Use serial number of reader to open ,then the “SerNum” parameter should set to this serial number.
		2	Use The detail USB driver path of the reader to open, then the “SerNum” parameter should set to this driver path.
4	SerNum	Serial number or driver path, when “AddrMode” value is 0, the parameter will be ignored.	

6.4 Build Connection String of Network

6.4.1 Format Example

"RDType=M201; CommType=NET; RemoteIP=192.168.0.246; RemotePort=9909;
LocalIP="

6.4.2 Parameter Description

No	Name	Parameter value	
1	RDType	Please refer to "Quick Start to Device Development"	
2	CommType	Communication interface type. Set "CommType=NET"	
		Value	Description
		COM	Serial port
		USB	USB interface
		NET	Network interface
		BLUETOOTH	Bluetooth interface
3	RemoteIP	IP address of destination reader	
4	RemotePort	Socket port opened for communication	
5	LocalIP	Local IP, can be used to bind the specified network card. Do not need to pass null values.	

6.5 Build Connection String of Bluetooth

6.5.1 Format Example

“RDType=RPAN;CommType=BLUETOOTH;Addr=00:0D:19:02:0C:DE”

6.5.2 Parameter Description

NO	Name	Parameter value	
1	RDType	Please refer to “Quick Start to Device Development”	
2	CommType	Communication interface type. Set “CommType=BLUETOOTH”	
		Value	Description
		COM	Serial port
		USB	USB interface
		NET	Network interface
		BLUETOOTH	Bluetooth interface
3	Addr	Bluetooth address	

7 Description of Common API Calls for The rfidlib_reader.dll

7.1 Enumerate serial ports

7.1.1 API Introduction

API Name	Description
COMPort_Enum	Enumerate the serial port installed in Windows and return the quantity. for detail, please see document "rfidlib_reader.doc"
COMPort_GetEnumItem	Obtain the serial port name of the enumerated item. for detail, please see document "rfidlib_reader.doc"

7.1.2 C# programming Example

```
comboBox1.Items.Clear();
UInt32 nCOMCnt = COMPort_Enum();
for ( i = 0; i < nCOMCnt; i++)
{
    StringBuilder comName = new StringBuilder();
    comName.Append("\0", 64);
    COMPort_GetEnumItem(i, comName, (UInt32)comName.Capacity);
    comboBox1.Items.Add(comName);
}
if (comboBox1.Items.Count > 0) comboBox1.SelectedIndex = 0;
```

7.2 Enumerate the Inserted USBHID Readers

7.2.1 API Introduction

API Name	Description
HID_Enum	Enumerate the inserted USBHID reader and return the quantity. for detail, please see document "rfidlib_reader.doc"
HID_GetEnumItem	Obtain the serial number or driver path of the enumerated item. for detail, please see document "rfidlib_reader.doc".

7.2.2 C# programming Example

```
comboBox9.Items.Clear();
UInt32 nCount = HID_Enum(driver.m_name);
int iret;
int i;
for ( i = 0; i < nCount; i++)
{
    StringBuilder sernum = new StringBuilder();
    sernum.Append("\0", 64);
    UInt32 nSize;
    nSize = (UInt32)sernum.Capacity;
    iret = HID_GetEnumItem((UInt32)i, HID_ENUM_INF_TYPE_SERIALNUM,
                           sernum,ref nSize);

    if (iret == 0)
    {
        comboBox9.Items.Add(sernum.ToString());
    }
}
```

7.3 Enumerate the Bluetooth Readers

7.3.1 API Introduction

API Name	Description
Bluetooth_Enum	Enumerates the matched Bluetooth reader and returns the quantity. For details, please see "rfidlib_reader.doc"
Bluetooth_GetEnumItem	Obtain the Bluetooth name or serial number of the enumerated item. for detail, please see document "rfidlib_reader.doc".

7.3.2 C# programming Example

```
UInt32 nBluetooth = Bluetooth_Enum();
for (UInt32 j = 0; j < nBluetooth; j++)
{
    StringBuilder nameBuf = new StringBuilder();
    StringBuilder addrBuf = new StringBuilder();
    UInt32 nSize = 256;
    nameBuf.Append("\0", (int)nSize);
    addrBuf.Append("\0", (int)nSize);
    Bluetooth_GetEnumItem(j, 1, nameBuf, ref nSize);
    nSize = 256;
    Bluetooth_GetEnumItem(j, 2, addrBuf, ref nSize);
    cbbBluetoothName.Items.Add(nameBuf);
    m_blueAddrList.Add(addrBuf.ToString());
}
```

7.4 Enumerates the Loaded RFID Reader Driver

7.4.1 API Introduction

API Name	Description
RDR_GetLoadedReaderDriverCount	Get the number of loaded reader drivers. For details, please see "rfidlib_reader .doc"
RDR_GetLoadedReaderDriverOpt	Get the parameter information of the reader driver. For details, please see "rfidlib_reader .doc"

7.4.2 C# Programming Example

Note: Before enumerating the loaded reader driver, make sure that the API "RDR_LoadReaderDrivers" is called.

```
UInt32 nCount;
nCount = RDR_GetLoadedReaderDriverCount(); //Get the quantity
uint i;
for( i=0;i<nCount;i++){
    UInt32 nSize;
    CReaderDriverInf driver = new CReaderDriverInf();
    StringBuilder strCatalog = new StringBuilder();
    strCatalog.Append("\0", 64);
    nSize = (UInt32)strCatalog.Capacity;
    //Get the class of the reader driver ,type can be reader class ,meeting gate class or library security
    gate class .
    RDR_GetLoadedReaderDriverOpt(i, LOADED_RDRDVR_OPT_CATALOG, strCatalog, ref nSize);
    driver.m_catalog = strCatalog.ToString();
    if (driver.m_catalog == RDRDVR_TYPE_READER) //is reader class ?
    {
        //Get name of the reader driver, such as "M201"
        StringBuilder strName = new StringBuilder();
        strName.Append("\0", 64);
        nSize = (UInt32)strName.Capacity;
        RDR_GetLoadedReaderDriverOpt(i, LOADED_RDRDVR_OPT_NAME, strName, ref nSize);
        driver.m_name = strName.ToString();
        //Get type coding of the reader driver, such as "680201"
        StringBuilder strProductType = new StringBuilder();
        strProductType.Append("\0", 64);
```

```

nSize = (UInt32)strProductType.Capacity;
RDR_GetLoadedReaderDriverOpt(i, LOADED_RDRDVR_OPT_ID, strProductType, ref nSize);
driver.m_productType = strProductType.ToString();
//Get supported communication port of the driver
StringBuilder strCommSupported = new StringBuilder();
strCommSupported.Append("\0", 64);
nSize = (UInt32)strCommSupported.Capacity;
RDR_GetLoadedReaderDriverOpt(i, LOADED_RDRDVR_OPT_COMMTYPESUPPORTED,
                             strCommSupported, ref nSize);
driver.m_commTypeSupported = (UInt32)int.Parse(strCommSupported.ToString());
readerDriverInfoList.Add(driver);
}
}

```


7.5 RS485 Bus programming

7.5.1 Description

RS485 bus uses the same serial port to communicate with multiple readers synchronously, so all the drivers of the reader of RS485 bus must share the same communication layer. First, open the corresponding reader driver with API "RDR_Open", and then call the API "**RDR_CreateRS485Node**" to create RS485 nodes, all the created RS485 Nodes share the same communication layer, the API description see "rfidlib_reader.doc".

7.5.2 C++ programming Example(ANRD201)

```
int iret ;
RFID_READER_HANDLE hANRD201;
RFID_READER_HANDLE hRS485_R1;
RFID_READER_HANDLE hRS485_R2;
RFID_READER_HANDLE hRS485_R3;

RDR_LoadReaderDrivers("\\Drivers");
//initial all RS485 Node to NULL
hANRD201= hRS485_R1= hRS485_R2= hRS485_R3 = NULL;
//Open RS485 bus communication layer
iret = RDR_Open("RDType=RD201;CommType=COM;COMName=COM5;BaudRate=38400;Frame=8E1;BusAddr=255,&hANRD201);
if(iret != 0) {
    // open fail;
    goto exit_func;
}
// Create first RS485 Node
iret = RDR_CreateRS485Node(hANRD201 , 1,&hRS485_R1) ;
if(iret !=0) {
    // create fail;
    goto exit_func;
}
//Create second RS485 node
iret = RDR_CreateRS485Node(hANRD201 , 2,&hRS485_R2) ;
```

```

if(iret !=0) {
    // create fail;
    goto exit_func;
}
//Create third RS485 node
iret = RDR_CreateRS485Node(hANRD201 , 3,&hRS485_R3) ;
if(iret !=0) {
    // create fail;
    goto exit_func;
}

//reader command process here .....

.....
// program exit
exit_func:
    //close first RS485 node
    if(hRS485_R1) {
        RDR_Close(hRS485_R1) ;
    }
    //close second RS485 node
    if(hRS485_R2) {
        RDR_Close(hRS485_R2) ;
    }
    //close third RS485 node
    if(hRS485_R3) {
        RDR_Close(hRS485_R3) ;
    }
    //Close communication layer
    if(hANRD201)
    {
        RDR_Close(hANRD201) ;
    }
}

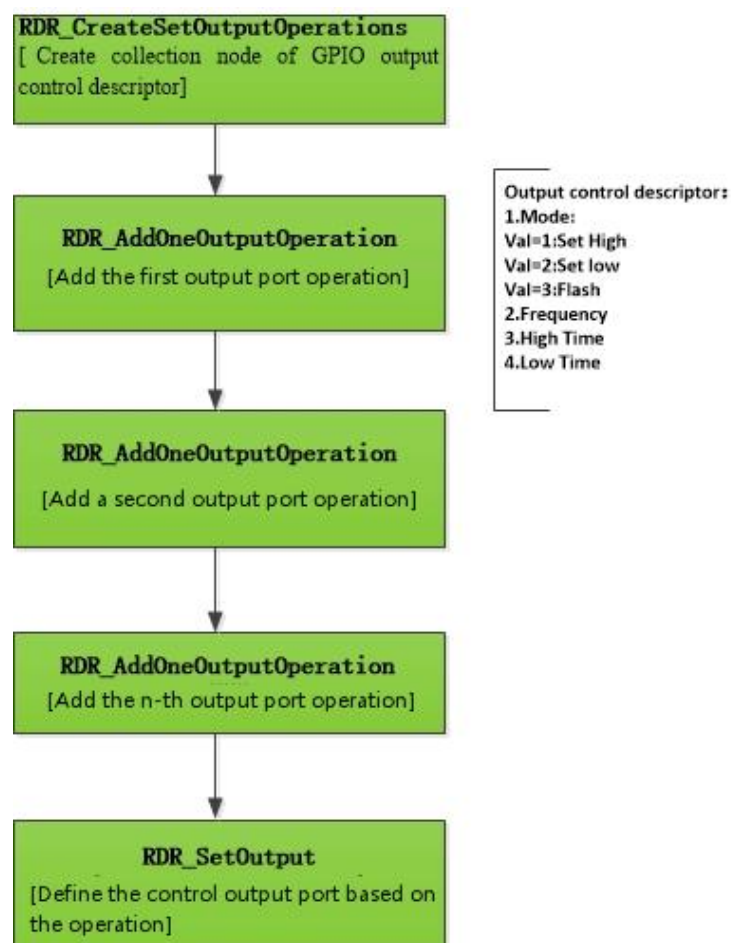
```

7.6 GPIO Output Control

7.6.1 API Introduction

API Name	Description
RDR_CreateSetOutputOperations	Create collection node of GPIO output control descriptor ,detail please see document"rfidlib_reader.doc"
RDR_AddOneOutputOperation	Add a new GPIO output control descriptor to collection node ,detail please see document"rfidlib_reader.doc"
RDR_SetOutput	A control command is sent to the RFID reader in accordance with the output control description

7.6.2 API Calling Process



6.6.3. C++ programming example

```
RFID_DN_HANDLE hOutputOperations;
BYTE outputNO; /* Output port Number,about output port number supported by reader please see reader
driver manual */
BYTE contrlMode ; /* Control mode: 1:Set high,2:set low,3:flash*/
BYTE frequency ; /*frequency of flash*/
BYTE activeDuration ; /* time for staying high */
BYTE inactiveDuration; /* time for staying low */
int iret;
outputNo = 1;
contrlMode=3;
frequency =2 ; /* flash two time */
activeDuration = 1 ;
inactiveDuration = 1;
hOutputOperations = RDR_CreateSetOutputOperations() ;
if(hOutputOperations) {
    RDR_AddOneOutputOperation(hOutputOperations ,
    outputNO, contrlMode, frequency, activeDuration, inactiveDuration) ;
    iret = RDR_SetOutput(hr , hOutputOperations) ;
    if(iret == 0) {
        /* Set output ok */
    }
    /* Release collection node */
    DNODE_Destroy (hOutputOperations) ;
}
```

7.7 Get Input GPIO Status

Call API `RDR_GetGPIOsStatus` to get input GPIO status. For detail, please see document “`rfidlib_reader.doc`”.

7.8 Get Device Information

Call API “`RDR_GetReaderInfor`” to get device information. For detail, please see document “`rfidlib_reader.doc`”.

7.9 Read/write Configuration Block

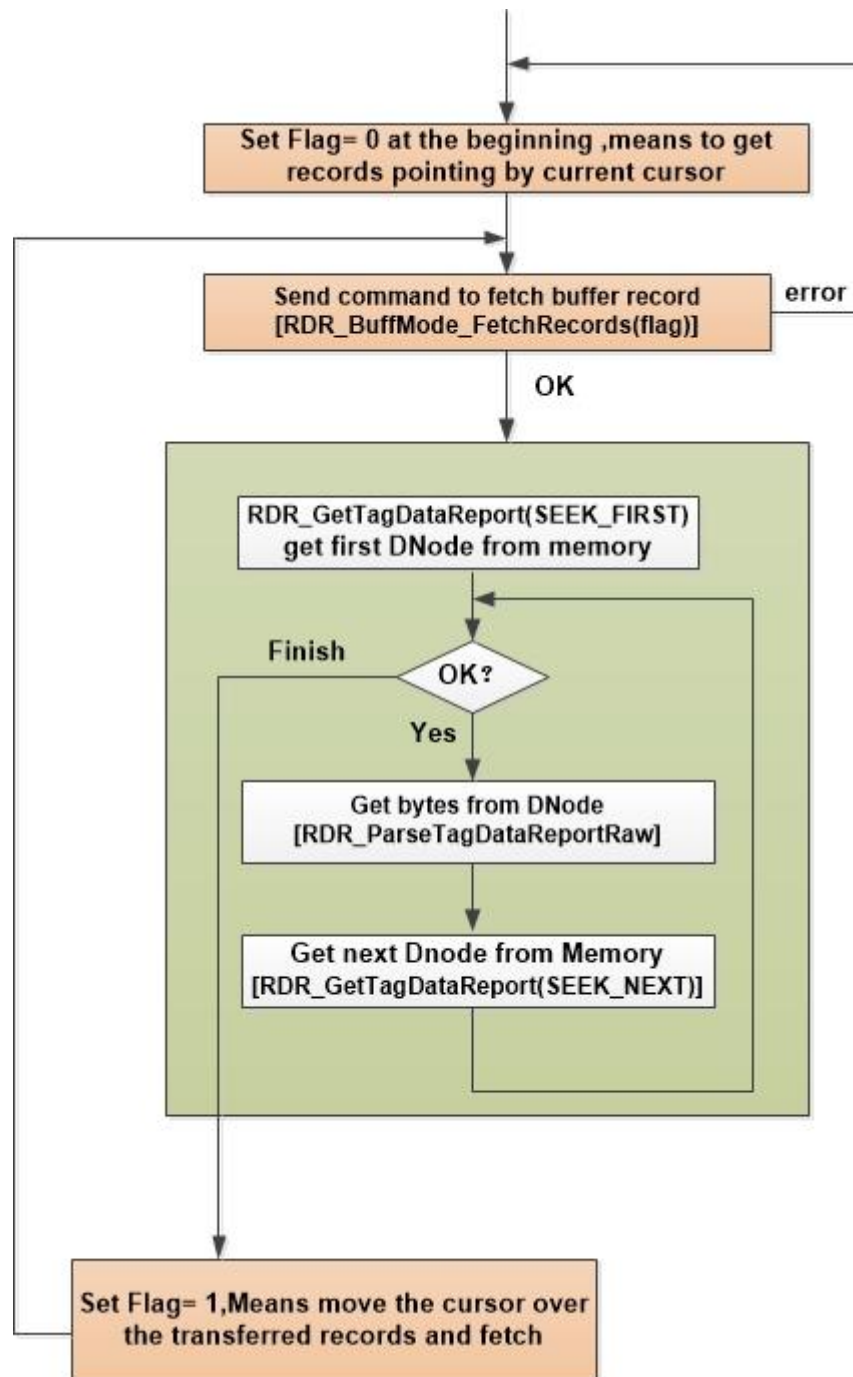
APIs:

1	<code>RDR_ConfigBlockWrite</code>	Write configuration block
2	<code>RDR_ConfigBlockRead</code>	Read configuration block
3	<code>RDR_ConfigBlockSave</code>	Save configuration block

For detail, please see document “`rfidlib_reader.doc`”.

7.10 To Fetch Buffer Record in Buffer Mode

This mode is supported by the active scanning mode of the handheld inventory-taker R-PAN to obtain records



Appendix 1. RFIDLIB Variable Type

1.1. Variable Type Description

Name	Length	Description
BOOLEAN	1	Boolean values, 0:FALSE,1:TRUE;
BYTE	1	Byte ,range is 0-255
WORD	2	Word, range is 0-65535
DWORD	4	Double word, range is 0- 4294967295
err_t	4	Integer
WPARAM	4 or 8	The first parameter value that the RFIDLIB API passes to the application via the SendMessage or callback function. In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
LPARAM	4 or 8	The second parameter value that the RFIDLIB API passes to the application via the SendMessage or callback function. In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
HWND	4 or 8	Window form handle. For details, see Windows API SendMessage. In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
WIN_MSG_t	4 or 8	Message value for the Window form. For details, see the Windows API SendMessage. In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
RFID_EVENT_CALLBACK	4 or 8	Pointer to call back function
LPCTSTR		String constant, used for application string input.
LPTSTR		String array, used for API string output
RFID_READER_HANDLE	4 or 8	RFID reader handle

		In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
RFID_TAG_HANDLE	4 or 8	RFID tag handle In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
RFID_DN_HANDLE	4 or 8	RFID data node handle In 32-bit operating systems, it should be defined as a 32-bit unsigned integer. In 64-bit operating systems, it should be defined as a 64-bit unsigned integer.
BYTE []	4 or 8	Byte array, use to input or output multiple bytes data.
BYTE *	4 or 8	Byte reference ,use to output single byte data
WORD *	4 or 8	Word reference ,use to output word data
DWORD *	4 or 8	Double word reference, use to output double word data.
RFID_READER_HANDLE *	4 or 8	RFID reader handle reference, use to output RFID reader handle.
RFID_TAG_HANDLE *	4 or 8	RFID tag handle reference, use to output tag handle.
RFID_DN_HANDLE *	4 or 8	RFID data node handle reference, use to output data node handle.

1.2. Variable Type Defined in C++

Name	Definition
BOOLEAN	unsigned char
BYTE	unsigned char
WORD	unsigned short
DWORD	unsigned int
err_t	int
WPARAM	32 bits: unsigned int 64 bits: unsigned __int64
LPARAM	32 bits: unsigned int 64 bits: unsigned __int64
HWND	32 bits: unsigned int 64 bits: unsigned __int64
WIN_MSG_t	32 bits: unsigned int 64 bits: unsigned __int64
RFID_EVENT_CALLBACK	VOID (__stdcall *RFID_EVENT_CALLBACK)(WPARAM wparam, LPARAM lparam) ;
LPCTSTR	ANSI: const char * ; unicode: const wchar * ;
LPTSTR	ANSI: char * ; unicode: wchar * ;
RFID_READER_HANDLE	32 bits: unsigned int 64 bits: unsigned __int64
RFID_TAG_HANDLE	32 bits: unsigned int 64 bits: unsigned __int64
RFID_DN_HANDLE	32 bits: unsigned int 64 bits: unsigned __int64
BYTE []	unsigned char variable[]
BYTE *	unsigned char *
WORD *	unsigned short *
DWORD *	unsigned int *
RFID_READER_HANDLE *	32 bits: unsigned int * 64 bits: unsigned __int64 *
RFID_TAG_HANDLE *	32 bits: unsigned int * 64 bits: unsigned __int64 *
RFID_DN_HANDLE *	32 bits: unsigned int * 64 bits: unsigned __int64 *

1.3. Variable Type Defined in C#

Name	Definition
BOOLEAN	Byte
BYTE	Byte
WORD	UInt16
DWORD	UInt32
err_t	int
WPARAM	UIntPtr
LPARAM	UIntPtr
HWND	UIntPtr
WIN_MSG_t	UInt32
RFID_EVENT_CALLBACK	Public delegate void RFIDLIB_EVENT_CALLBACK(UIntPtr wparam, UIntPtr lparam);
LPCTSTR	string
LPTSTR	StringBuilder
RFID_READER_HANDLE	UIntPtr
RFID_TAG_HANDLE	UIntPtr
RFID_DN_HANDLE	UIntPtr
BYTE []	Byte[]
BYTE *	ref Byte
WORD *	ref UInt16
DWORD *	ref UInt32
RFID_READER_HANDLE *	ref UIntPtr
RFID_TAG_HANDLE *	ref UIntPtr
RFID_DN_HANDLE *	ref UIntPtr

1.4. Variable Type Defined in Delphi(only WIN32 platform)

Name	Definition
BOOLEAN	byte
BYTE	byte
WORD	word
DWORD	longword
err_t	integer
WPARAM	longword
LPARAM	longword
HWND	longword
WIN_MSG_t	longword
RFID_EVENT_CALLBACK	procedure(wParam: longword; lParam: longword) ; stdcall;
LPCTSTR	string
LPTSTR	pchar
RFID_READER_HANDLE	longword
RFID_TAG_HANDLE	longword
RFID_DN_HANDLE	longword
BYTE []	PBYTE ,type PBYTE=^byte ;
BYTE *	var byte
WORD *	var word
DWORD *	var longword
RFID_READER_HANDLE *	var longword
RFID_TAG_HANDLE *	var longword
RFID_DN_HANDLE *	var longword

1.5. Variable Type Defined in VB

Name	Definition
BOOLEAN	Byte
BYTE	Byte
WORD	Integer
DWORD	Long
err_t	Long
WPARAM	Long
LPARAM	Long
HWND	Long
WIN_MSG_t	Long
RFID_EVENT_CALLBACK	
LPCTSTR	String
LPTSTR	Fixed length String, Dim str1 As String * n(n is max character count to store)
RFID_READER_HANDLE	long
RFID_TAG_HANDLE	long
RFID_DN_HANDLE	long
BYTE []	ByRef x as byte
BYTE *	ByRef x as byte
WORD *	ByRef x as integer
DWORD *	ByRef x as long
RFID_READER_HANDLE *	ByRef x as long
RFID_TAG_HANDLE *	ByRef x as long
RFID_DN_HANDLE *	ByRef x as long

1.6. Variable Type Defined in VB.net

Name	Definition
BOOLEAN	Byte
BYTE	Byte
WORD	UInt16
DWORD	UInt32
err_t	integer
WPARAM	UIntPtr
LPARAM	UIntPtr
HWND	UIntPtr
WIN_MSG_t	UInt32
RFID_EVENT_CALLBACK	Public delegate Sub RFIDLIB_EVENT_CALLBACK(byval wParam as UIntPtr, byval lParam as UIntPtr)
LPCTSTR	String
LPTSTR	StringBuilder
RFID_READER_HANDLE	UIntPtr
RFID_TAG_HANDLE	UIntPtr
RFID_DN_HANDLE	UIntPtr
BYTE []	ref Byte, passed to the first element of the array XXX(0) as an argument
BYTE *	ref Byte
WORD *	ref UInt16
DWORD *	ref UInt32
RFID_READER_HANDLE *	ref UIntPtr
RFID_TAG_HANDLE *	ref UIntPtr
RFID_DN_HANDLE *	ref UIntPtr

Appendix 2. Air Protocol Type and Tag Type Coding

2.1. Air protocol Type Code Definition

Type	Code	Memo
HF ISO15693	1	HF ISO15693 Air interface standard
HF ISO14443A	2	HF ISO15693 Air interface standard

2.2. Tag Type Code Definition

2.2.1. ISO15693

Tag Type	ID	Memo
NXP ICODE SLI	1	SL2S2001
Tag-it HF-I plus	2	
ST M24LRxx	3	ST M24LR Series
Fujitsu MB89R118C	4	
ST M24LR64	5	
ST M24LR16E	6	
NXP ICODE SLIX	7	SL2S2002 / SL2S2102
Tag-it HF-I Standard	8	
Tag-it HF-I Pro	9	

2.2.2. ISO14443A

Tag Type	ID	
Mifare ultralight 1K and ultralight C	1	
Mifare S50	2	
Mifare S70	3	
NXP DESFIRE EV1	4	
NTAG21x	5	
MIFARE MINI	6	
MIFARE_PLUS	7	

Appendix 3. RFIDLIB API Error Table

Error	Description
0	No error, the API is successfully invoked.
-1	Unknown error
-2	Communication with the card reader hardware failed
-3	The API passed in an incorrect parameter
-4	The value of the parameter passed to the API is not supported, such as the parameter value can only be 0-5, if 6 is passed then the error will be returned.
-5	Timeout, the command sent to the card reader, the data cannot be returned within the set time.
-6	API failed to request memory
-7	The function is disabled or the link is incorrectly connected
-8	-ERR_BUSY: indicates that the command is busy
-9	Reserved
-10	Reserved
-11	-ERR_SIZE: The message length is incorrect.
-12	The packet length returned by the card reader is incorrect
-13	Reserved
-14	-ERR_CLOSED: The communication resource is closed.
-15	Reserved
-16	- ERR_LOWER version is too low.
-17	Error from reader, can use "RDR_GetReaderLastReturnError" to get reader error code .
-18	ERR_CHECKSUM: The checksum part is incorrect and the communication frame is invalid
-19	ERR_USED: indicates that the communication resource is occupied.
-20	Reserved
-21	Inventory stop trigger occurs. For example, if 1 second is set as the maximum reading time for Inventory, the reader will terminate Inventory if all the labels are not read within 1 second, and the API will return this error to tell the application that there may still be labels not read.
-22	Invalid tag command
-23	Invalid Configuration items passed in RDR_SetConfig or RDR_GetConfig
-24	Reserved

-25	TCP socket error. API returns this error indicating that the TCP connection has been disconnected.
-26	The buffer inputted by the application layer is too small.
-27	The data returned by the card reader is incorrect.
-28	Reserved
-29	Reserved
-30	-ERR_WSAEWOULDBLOCK, no connection established
-31	-ERR_WSAESHUTDOWN, need to block to check the connection or send/receive status
-32	-ERR_WSAESHUTDOWN, the Ethernet interface is disconnected
-33	-ERR_NOCONN. No connection is established
-34	Reserved
-35	-ERR_QUICKEXIT, this error indicate the calling API quick exit from command execution.
-36	-ERR_ANTENNA_PARAMETER , indicates an antenna parameter error warning or temperature warning
-37	-ERR_CFGBLOCKLOCK, the reader configuration block is locked
-38	-ERR_TRANSPORTRECEIVE , data transparent transmission to tag failed
-39	-ERR_NOTONSHELF, not on shelf

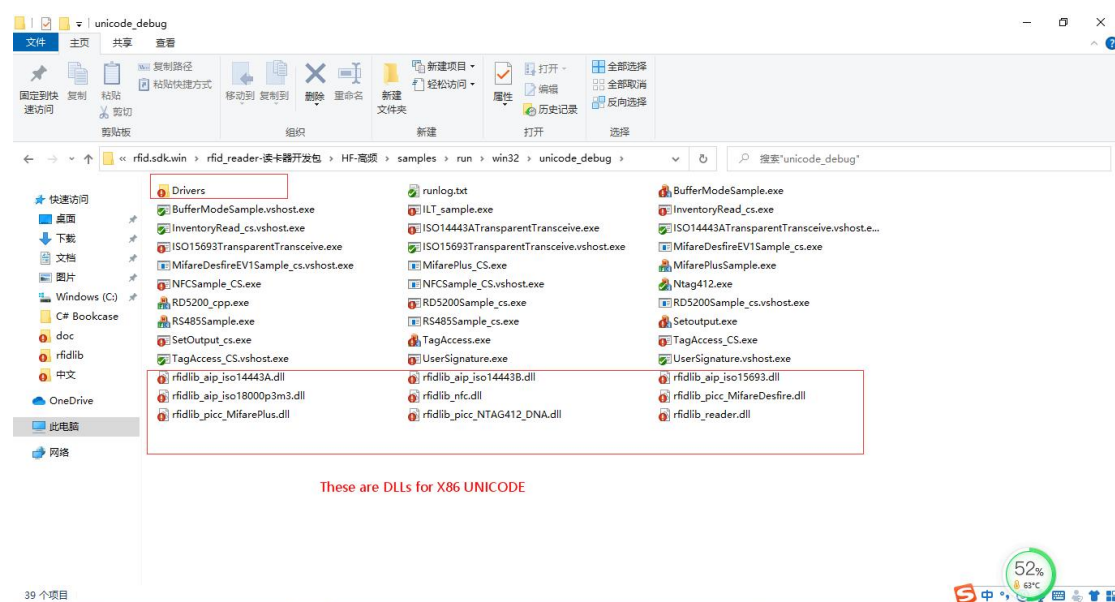
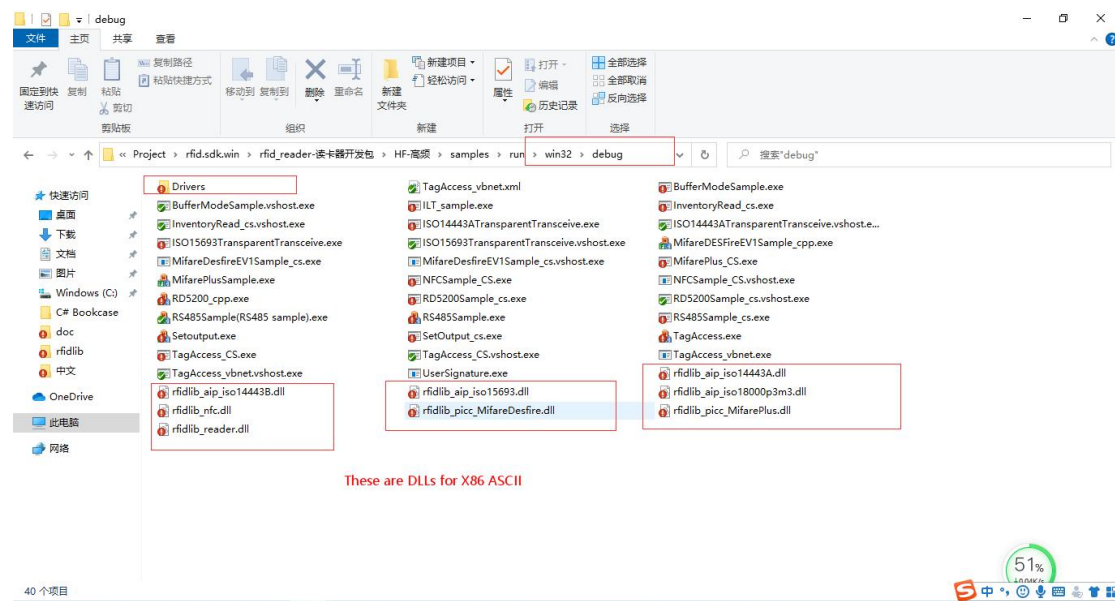
Appendix 4: Q&A

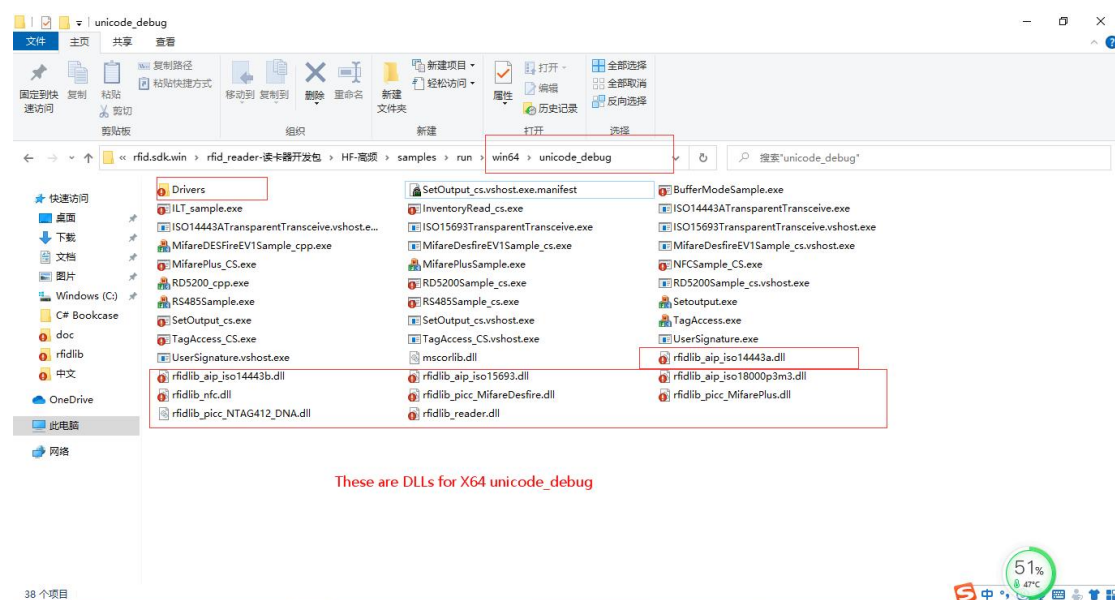
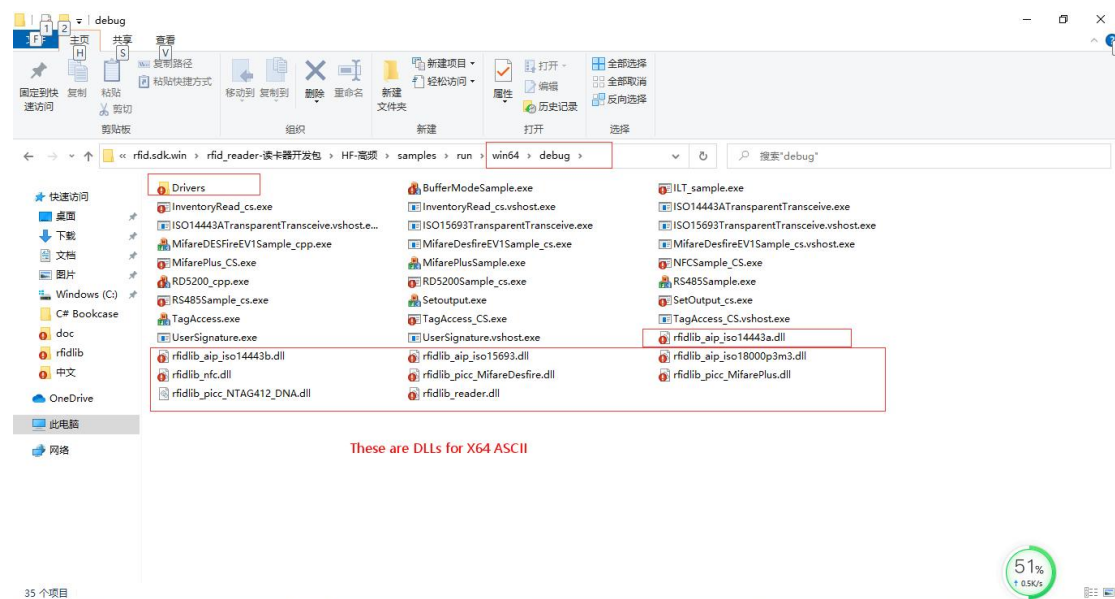
1Q:

Are the DLLs for readers divided into four types: x86 ascii, x86 unicode, x64 ascii, and x64 unicode?

1A:

Yes, due to the presence of characters in our API, and compiled in conjunction with the CPU architecture, so our DLLS are divided into x86 ascii, x86 unicode, x64 ascii, and x64 unicode dll.





2Q:

Can different versions of DLLs be mixed?

2A:

It is best not to mix different versions of DLLs, for example, the DLL of rfid.sdk.win v02.12 should not be mixed with the DLL of rfid.sdk.win v02.13.

You can replace the original DLL of rfid.sdk.win v02.12 with the DLL of rfid.sdk.win v02.13.

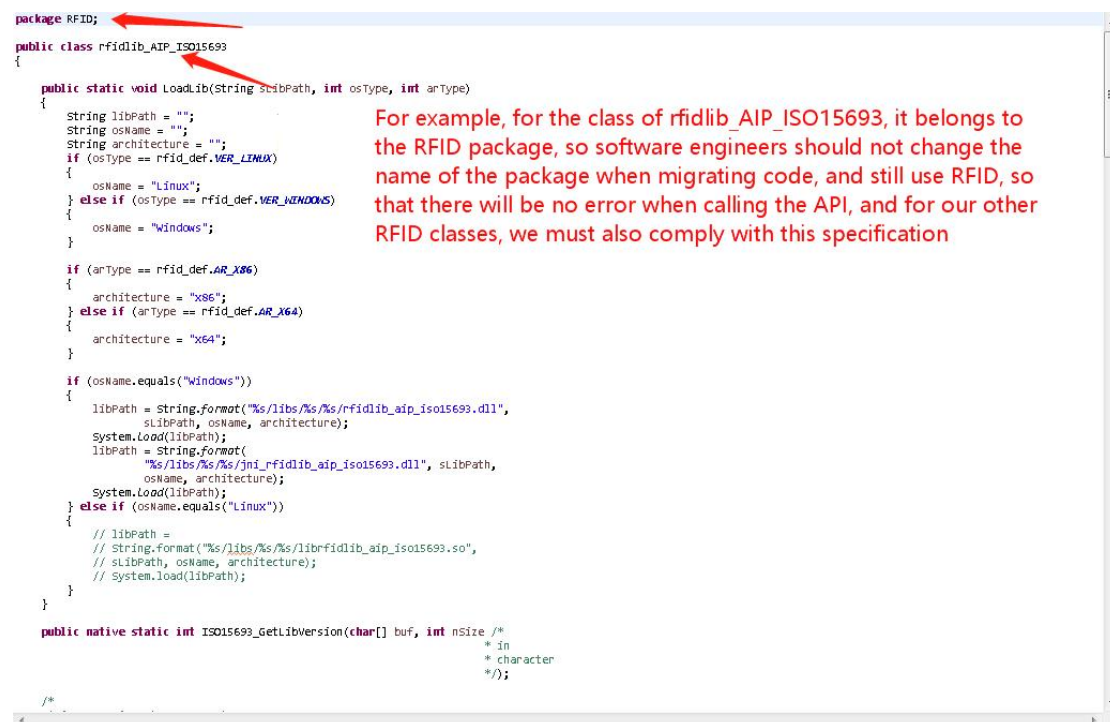
3Q:

Can I modify the RFID project package name for JAVA projects?

3A:

No, because some of our JAVA DLLs are generated by JNI, which relies on package names. Therefore, JAVA software engineers should still migrate the code according to the package names in our DEMO when writing your own applications.

As shown in the figure:



4Q:

Do I need to switch antennas for reading and writing tags?

4A:

Not required for single antenna readers, but for multi antenna readers, it is necessary to call RDR_SetAccessAntenna (the antenna where the tag is located) to switch the antenna before operating the antenna.

For example, now there are two 15693 tags, the first tag is located at antenna 1 and the second tag is located at antenna 2. Now I need to read the tag data block, and the process should be as follows:

RDR_SetAccessAntenna(Antenna1)--->ISO15693_Connect(address mode, first tag UID)-> Read Block ->RDR_TagDisconnect()->RDR_SetAccessAntenna(Antenna 2)--> ISO15693_Connect(address mode, second tag UID)-> Read Block ->RDR_TagDisconnect()

Other writing blocks and modifying AFI should follow a similar process.

5Q:

How to select RDType in the reader connection string?

5A:

Please refer to "Device Development Quickstart.doc" for device driver selection.

6Q:

How to gracefully and securely close reader communication connections?

6A:

It is realized by RDR_SetCommulmmeTimeout and RDR_ResetCommulmmeTimeout.

And then causes the called function in the blocked wait state to exit immediately. For detailed usage of these two functions, refer to the calls to these functions in TagAccess, TagAccess_CS.

7Q:

The API call process is correct, and the connection string is also correct, but the API RDR_Open returns -4;

7A:

This situation is mainly that the program project does not define UNICODE, but it uses the dll of UNICODE, and replacing the correct DLLs can solve the problem.

8Q:

Software startup did not call RDR_LoadReaderDrivers causing RDR_Open returns -3.